

CHRIST (Deemed to be University)
Department of Computer Science
Master of Computer Applications

Course: MCA411-2 - PYTHON Programming

LAB Exercise II –

Demonstrate Custom modules with functions.

Date of assignment: 21-09-2026

Date of submission: 21-09-2026

Name : Aditya Kumar

Reg. No : 2647203

Class : 2 MCA – B

Domain : Clean Energy Systems & Grid Decarbonization

Lab 2

Lab Exercise II:

Demonstrate Custom modules with functions.

- A. Design a reusable domain-independent utility modules and demonstrate its integration into your application.

Code:

```
'''
# Reusable Domain-Independent Utility Modules
'''

# Prints a Title Banner in between '='
def print_title(msg):
    print("\n" + "=" * 45)
    print(msg)
    print("=" * 45)

# Prints a simple line divider
def print_divider():
    print("-" * 45)

# Checks if a string is non-empty
def check_empty(text):
    return text.strip() != ""

# Formats the Float with 2 Decimal places
def format_decimal(val):
    return f"{float(val):.2f}"

# Check if the string enter is a no or not
def check_num(val_str):
    return val_str.strip().replace(".", "", 1).isdigit()

# Takes confirmation before Proceeding
def confirm_action(prompt_text):
    ans = input(f"{prompt_text} (y/n): ").strip().lower()
    return ans == "y"

# Checks if the Value is in the reange or not
def check_range(val, min_val, max_val):
    return min_val <= val <= max_val
```

Output:

print_title:

```
=====
SOLAR-BESS OPERATIONS CONSOLE
=====
1. View All Sites & Thermal Diagnostics
2. Log Power Reading & Performance Ratio
3. Search Site Record by ID
4. System Operation & Energy Analysis
5. Exit
```

print_divider:

```
=====
ALL REGISTERED MICROGRID SITES
=====
[SITE_1] Location: Substation North | Rated Capacity: 150.0 kW
        Battery SoC: 75.00% | Cell Temp: 32.00°C [TEMP_NORMAL]
        Power Readings (kW): [40.0, 55.0, 65.0]
        Average Power: 53.33 kW | Peak Recorded: 65.0 kW
-----
[SITE_2] Location: Substation South | Rated Capacity: 90.0 kW
        Battery SoC: 25.00% | Cell Temp: 48.00°C [OVERHEAT_WARNING]
        Power Readings (kW): [20.0, 30.0, 25.0]
        Average Power: 25.0 kW | Peak Recorded: 30.0 kW
-----
```

format_decimal:

```
-----
[SITE_1] Location: Substation North | Rated Capacity: 150.0 kW
        Battery SoC: 75.00% | Cell Temp: 32.00°C [TEMP_NORMAL]
        Power Readings (kW): [40.0, 55.0, 65.0]
        Average Power: 53.33 kW | Peak Recorded: 65.0 kW
-----
```

confirm_action:

```
=====
LOG REAL-TIME POWER READING
=====
Enter Site ID: Site_1
Enter current generated power in kW (0 to 500): 250
Calculated Performance Ratio (PR): 100.0%
Confirm logging this reading? (y/n): y
Success: Appended 250.00 kW to SITE_1.
```

check_range:

```
=====
LOG REAL-TIME POWER READING
=====
Enter Site ID: site_1
Enter current generated power in kW (0 to 500): 1000
Validation Error: Reading must be between 0.0 and 500.0 kW.
```

check_num:

```
=====
LOG REAL-TIME POWER READING
=====
Enter Site ID: site_1
Enter current generated power in kW (0 to 500): aaa
Validation Error: Input must be a valid number.
```

B. Create a custom Python module containing at least five reusable functions relevant to your selected domain.

Code:

```
'''
# Reusable Domain-Based Modules
'''

# Add power reading to the History List
def add_reading(readings_list, new_kw):
    readings_list.append(round(new_kw, 2))

# Calculate Average Readings for the Site
def cal_average(readings_list):
    if len(readings_list) == 0:
        return 0.0
    return round(sum(readings_list) / len(readings_list), 2)

# Find Highest (peak) Reading
def get_peak(readings_list):
    if len(readings_list) == 0:
        return 0.0
    return max(readings_list)

# Search site using ID
def search_site(sites_dict, site_id):
    return sites_dict.get(site_id.strip().upper())

# Calculate net power surplus or deficit
def calculate_balance(solar_kw, load_kw):
    net = solar_kw - load_kw
    if net > 0:
        status = "SURPLUS"
    elif net < 0:
        status = "DEFICIT"
    else:
        status = "BALANCED"
    return round(net, 2), status

# Check if battery temperature is within safe limits
```

```

def check_thermal_status(temp_c, max_limit=45.0):
    if temp_c > max_limit:
        return False, "OVERHEAT_WARNING"
    return True, "TEMP_NORMAL"

# Calculate solar performance ratio (PR)
def cal_pr(actual_kw, rated_capacity_kw):
    # Computes system efficiency ratio: (Actual Power / Rated Capacity) * 100.
    if rated_capacity_kw <= 0:
        return 0.0
    pr = (actual_kw / rated_capacity_kw) * 100
    if pr > 100.0:
        pr = 100.0
    return round(pr, 2)

```

Output:

add_reading:

```

Enter choice (1-5): 2

=====
LOG REAL-TIME POWER READING
=====
Enter Site ID: Site_1
Enter current generated power in kW (0 to 500): 250
Calculated Performance Ratio (PR): 100.0%
Confirm logging this reading? (y/n): y
Success: Appended 250.00 kW to SITE_1.

```

cal_average:

```

=====
[SITE_1] Location: Substation North | Rated Capacity: 150.0 kW
        Battery SoC: 75.00% | Cell Temp: 32.00°C [TEMP_NORMAL]
        Power Readings (kW): [40.0, 55.0, 65.0]
        Average Power: 53.33 kW | Peak Recorded: 65.0 kW
=====

```

get_peak:

```

=====
[SITE_1] Location: Substation North | Rated Capacity: 150.0 kW
        Battery SoC: 75.00% | Cell Temp: 32.00°C [TEMP_NORMAL]
        Power Readings (kW): [40.0, 55.0, 65.0]
        Average Power: 53.33 kW | Peak Recorded: 65.0 kW
=====

```

search_site:

```
=====
SEARCH SITE DETAILS
=====
Enter Site ID: Site_1

Site ID : SITE_1
Location : Substation North
Rated Capacity : 150.0 kW
Battery SoC : 75.00%
Cell Temp : 32.00°C (TEMP_NORMAL)
Power Log (kW) : [40.0, 55.0, 65.0, 250.0]
Average Power : 102.5 kW
Peak Power : 250.0 kW
```

calculate_balance:

```
Enter choice (1-5): 4

=====
System Operation & Energy Analysis
=====
Enter Site ID: Site_2
Enter current Solar Generation (kW): 500
Enter current Grid Load Demand (kW): 600

Operational Status : DEFICIT
Net Power Flow      : -100.00 kW
Asset Efficiency     : 100.0% PR
[ALERT]: Unit core temperature is high (48.0°C)! Dispatch with caution.
```

check_thermal_status:

```
Enter choice (1-5): 4

=====
System Operation & Energy Analysis
=====
Enter Site ID: Site_2
Enter current Solar Generation (kW): 500
Enter current Grid Load Demand (kW): 600

Operational Status : DEFICIT
Net Power Flow      : -100.00 kW
Asset Efficiency     : 100.0% PR
[ALERT]: Unit core temperature is high (48.0°C)! Dispatch with caution.
```

cal_pr:

```
Enter choice (1-5): 2

=====
LOG REAL-TIME POWER READING
=====
Enter Site ID: Site_1
Enter current generated power in kW (0 to 500): 250
Calculated Performance Ratio (PR): 100.0%
Confirm logging this reading? (y/n): y
Success: Appended 250.00 kW to SITE_1.
```

Main Python File Code:

```
import Module_2647203_P2 as mod

# Operational Dataset
sites = {
    "SITE_1": {
        "location": "Substation North",
        "capacity_kw": 150.0,
        "soc": 75.0,
        "temp": 32.0,
        "readings": [40.0, 55.0, 65.0]
    },
    "SITE_2": {
        "location": "Substation South",
        "capacity_kw": 90.0,
        "soc": 25.0,
        "temp": 48.0,
        "readings": [20.0, 30.0, 25.0]
    },
    "SITE_3": {
        "info": ("Substation East", 200.0),
        "location": "Substation East",
        "capacity_kw": 200.0,
        "soc": 85.0,
        "temp": 28.0,
        "readings": [80.0, 95.0, 110.0]
    }
}

# Print All the Added Sites
def view_all_sites():
    mod.print_title("ALL REGISTERED MICROGRID SITES")
    for s_id, d in sites.items():
        avg_kw = mod.cal_average(d["readings"])
        peak_kw = mod.get_peak(d["readings"])
        is_safe, thermal_status = mod.check_thermal_status(d["temp"])

        print(f"[{s_id}] Location: {d['location']} | Rated Capacity: {d['capacity_kw']} kW")
```

```

        print(f"\t Battery SoC: {mod.format_decimal(d['soc'])}% | Cell Temp:
{mod.format_decimal(d['temp'])}°C [{thermal_status}]")
        print(f"\t Power Readings (kW): {d['readings']}")
        print(f"\t Average Power: {avg_kw} kW | Peak Recorded: {peak_kw} kW")
        mod.print_divider()

# Add new reading for a Site
def add_power_reading():
    mod.print_title("LOG REAL-TIME POWER READING")
    s_id = input("Enter Site ID: ").strip().upper()
    site = mod.search_site(sites, s_id)

    if not site:
        print("Error: Site ID does not exist.")
        return

    val_str = input("Enter current generated power in kW (0 to 500): ").strip()

    # Using module validation utilities
    if not mod.check_num(val_str):
        print("Validation Error: Input must be a valid number.")
        return

    val = float(val_str)
    if not mod.check_range(val, 0.0, 500.0):
        print("Validation Error: Reading must be between 0.0 and 500.0 kW.")
        return

    # Check performance ratio before saving
    pr = mod.cal_pr(val, site["capacity_kw"])
    print(f"Calculated Performance Ratio (PR): {pr}%")

    if mod.confirm_action("Confirm logging this reading?"):
        mod.add_reading(site["readings"], val)
        print(f"Success: Appended {mod.format_decimal(val)} kW to {s_id}.")
    else:
        print("Action cancelled.")

# Searching a Site
def search_site_record():
    mod.print_title("SEARCH SITE DETAILS")
    s_id = input("Enter Site ID: ").strip().upper()
    site = mod.search_site(sites, s_id)

    if site:
        avg_kw = mod.cal_average(site["readings"])
        peak_kw = mod.get_peak(site["readings"])
        is_safe, thermal_status = mod.check_thermal_status(site["temp"])

        print("\n")
        print(f"Site ID : {s_id}")
        print(f"Location : {site['location']}")
        print(f"Rated Capacity : {site['capacity_kw']} kW")
        print(f"Battery SoC : {mod.format_decimal(site['soc'])}%")
        print(f"Cell Temp : {mod.format_decimal(site['temp'])}°C
({thermal_status})")
        print(f"Power Log (kW) : {site['readings']}")
        print(f"Average Power : {avg_kw} kW")
        print(f"Peak Power : {peak_kw} kW")

```

```

else:
    print("Error: Site record not found.")

def simulate_dispatch():
    mod.print_title("System Operation & Energy Analysis")
    s_id = input("Enter Site ID: ").strip().upper()
    site = mod.search_site(sites, s_id)

    if not site:
        print("Error: Site record not found.")
        return

    sol_str = input("Enter current Solar Generation (kW): ").strip()
    load_str = input("Enter current Grid Load Demand (kW): ").strip()

    if not (mod.check_num(sol_str) and mod.check_num(load_str)):
        print("Validation Error: Both solar and load inputs must be numbers.")
        return

    sol = float(sol_str)
    load = float(load_str)

    # Calculate net balance using module function
    net, status = mod.calculate_balance(sol, load)
    pr = mod.cal_pr(sol, site["capacity_kw"])

    print(f"\nOperational Status : {status}")
    print(f"Net Power Flow      : {mod.format_decimal(net)} kW")
    print(f"Asset Efficiency     : {pr}% PR")

    # Warn if battery cell temperature is high
    is_safe, thermal_status = mod.check_thermal_status(site["temp"])
    if not is_safe:
        print(f"[ALERT]: Unit core temperature is high ({site['temp']}°C)!
Dispatch with caution.")

while True:
    mod.print_title("SOLAR-BESS OPERATIONS CONSOLE")
    print("1. View All Sites & Thermal Diagnostics")
    print("2. Log Power Reading & Performance Ratio")
    print("3. Search Site Record by ID")
    print("4. System Operation & Energy Analysis")
    print("5. Exit")

    choice = input("\nEnter choice (1-5): ").strip()

    if choice == "1":
        view_all_sites()
    elif choice == "2":
        add_power_reading()
    elif choice == "3":
        search_site_record()
    elif choice == "4":
        simulate_dispatch()
    elif choice == "5":
        print("Terminating console application.")
        break
    else:

```

```
print("Invalid selection. Please choose 1 to 5.")
```

Output:

Main Menu:

```
=====
SOLAR-BESS OPERATIONS CONSOLE
=====
1. View All Sites & Thermal Diagnostics
2. Log Power Reading & Performance Ratio
3. Search Site Record by ID
4. System Operation & Energy Analysis
5. Exit

Enter choice (1-5): |
```

List all site:

```
Enter choice (1-5): 1

=====
ALL REGISTERED MICROGRID SITES
=====
[SITE_1] Location: Substation North | Rated Capacity: 150.0 kW
        Battery SoC: 75.00% | Cell Temp: 32.00°C [TEMP_NORMAL]
        Power Readings (kW): [40.0, 55.0, 65.0]
        Average Power: 53.33 kW | Peak Recorded: 65.0 kW
-----
[SITE_2] Location: Substation South | Rated Capacity: 90.0 kW
        Battery SoC: 25.00% | Cell Temp: 48.00°C [OVERHEAT_WARNING]
        Power Readings (kW): [20.0, 30.0, 25.0]
        Average Power: 25.0 kW | Peak Recorded: 30.0 kW
-----
[SITE_3] Location: Substation East | Rated Capacity: 200.0 kW
        Battery SoC: 85.00% | Cell Temp: 28.00°C [TEMP_NORMAL]
        Power Readings (kW): [80.0, 95.0, 110.0]
        Average Power: 95.0 kW | Peak Recorded: 110.0 kW
-----
```

Log Reading and Performance Ratio:

```
Enter choice (1-5): 2

=====
LOG REAL-TIME POWER READING
=====
Enter Site ID: Site_1
Enter current generated power in kW (0 to 500): 250
Calculated Performance Ratio (PR): 100.0%
Confirm logging this reading? (y/n): y
Success: Appended 250.00 kW to SITE_1.
```

Search:

```
Enter choice (1-5): 3

=====
SEARCH SITE DETAILS
=====
Enter Site ID: Site_1

Site ID : SITE_1
Location : Substation North
Rated Capacity : 150.0 kW
Battery SoC : 75.00%
Cell Temp : 32.00°C (TEMP_NORMAL)
Power Log (kW) : [40.0, 55.0, 65.0, 250.0]
Average Power : 102.5 kW
Peak Power : 250.0 kW
```

System Operation & Energy Analysis:

```
Enter choice (1-5): 4

=====
System Operation & Energy Analysis
=====
Enter Site ID: Site_2
Enter current Solar Generation (kW): 500
Enter current Grid Load Demand (kW): 600

Operational Status : DEFICIT
Net Power Flow      : -100.00 kW
Asset Efficiency     : 100.0% PR
[ALERT]: Unit core temperature is high (48.0°C)! Dispatch with caution.
```